

Тема: «Внутренние объекты Array, Date, Math, String»

Цель: ознакомиться с внутренними объектами языка сценариев JavaScript, изучить синтаксис объектов Array, Date, Math, String, проверить выполнение сценариев с использованием предопределенных объектов, самостоятельно разработать сценарии с применением различных свойств и методов внутренних объектов.

Время выполнения: 4 часа.

1 Внутренние объекты

Внутренние объекты не относятся к браузеру или загруженному в настоящее время HTML-документу. Эти объекты могут создаваться и обрабатываться в любой JavaScript-программе. Они включают в себя простые типы, такие как строки, а также более сложные объекты, в частности даты.

Имя объекта	Описание
Array	Массив. Не поддерживается в браузерах старых версий
Date	Дата и время
Math	Поддержка математических функций
Object	Обобщенный объект. Не поддерживается в старых версиях
String	Текстовая строка. Не поддерживается в браузерах старых версий

1.1 Объект Array

Array – это упорядоченное множество объектов. Обращение к объектам ведется при помощи целочисленного индекса. Примерами объектов-массивов в браузере служат гиперсвязи, метки, формы, фреймы. Массив можно создать одним из следующих способов:

- 1) используя определенную пользователем функцию для присвоения объекту многих значений;
- 2) используя конструктор Array();
- 3) используя конструктор Object().

Методы и свойства объекта Array() представлены в таблице 1.

Таблица 1 – Методы и свойства объекта Array()

Тип	Элемент	Описание
1	2	3
Метод	concat()	Соединяет элементы существующего массива.
	join()	Соединяет все элементы массива в одну строку.
	pop()	Удаляет последний элемент массива.
	push()	Добавляет элементы в конец массива.
	reverse()	Изменяет порядок следования элементов в массиве.
	shift()	Удаляет элементы в начале массива.
	slice()	Возвращает часть массива.
	sort()	Сортирует элементы в массиве.
	splice()	Вставляет и удаляет элементы из массива.
	map()	возвращает переданные функцией элементы нового массива
	toString()	Преобразует элементы массива в строку.

	unshift()	Добавляет элементы в начало массива.
	valueOf()	Возвращает исходный массив элементов.
1	2	3
Свойство	index	Возвращает индекс совпадения в строке для массива, созданного в соответствии с регулярным выражением.
	input	Для массива, созданного в соответствии с регулярным выражением, свойство возвращает исходную строку.
	length	Количество элементов в массиве.
	prototype	Позволяет добавлять свойства к экземплярам объекта Array.

Ниже представлен пример программы вывода на экран элементов массива, представляющих собой название дней недели.

Листинг 1.1 Вывод на экран элементов массива, представляющих собой название дней недели

```

<!DOCTYPE html>
<html>
<head>
<title>Array</title>
</head>
<body>
<script>

days=new Array()
days[0]="воскресенье"
days[1]="понедельник"
days[2]="вторник"
days[3]="среда"
days[4]="четверг"
days[0]="пятница"
days[10]="XXXXXX"
document.write(days)
for(i=0;i<days.length;i++){
document.write(days[i], "<br>")
}
</script>
</body>
</html>

```

Объясните работу приведенного ниже сценария.

Листинг 1.2 Название

```

<!DOCTYPE html>
<html>
<head>
<title>Array </title>
<script>
day=new Array()
day[0]="voskresenie"
day[1]="ponedelnik"
day[2]="vtornik"
day[3]="3da"
day[4]="4etveg"

```

```

day[5]="5nizza"
day[6]="subbota"
</script>
</head>
<body>
<script>
document.write(day.length,"<br>")
document.write(day,"<br>")
</script>
</body>
</html>

```

1.2 Объект Date

Объект содержит информацию о дате и времени. Этот объект имеет множество методов, предназначенных для получения такой информации. Кроме того объекты Date можно создавать и изменять, например, путем сложения или вычитания значений дат получать новую дату.

Для создания объекта Date применяется следующий синтаксис:

```
dateObj = new Date(parameters)
```

где dateObj – переменная, в которую будет записан новый объект Date. Аргумент parameters может принимать следующие значения:

- 1) пустой параметр, например, date() в данном случае дата и время – текущие системные;
- 2) строку, представляющую дату и время в виде: «месяц, день, год, время», например, “March 1, 2000, 17:00:00”. Время представлено в 24-часовом формате;
- 3) значения года, месяца, дня, часа, мину, секунд. Например, строка “00,4,1,12,30,0” означает 1 апреля 2000 года, 12.30.
- 4) целочисленные значения только для года, месяца, и дня, например, “00,5,1” означает 1 мая 2000 года, сразу после полночи, так как значения времени равны нулю.

Методы и свойства объекта Date представлены в таблице 2.

Таблица 2 –Методы и свойства объекта Date

Тип	Элемент	Описание
1	2	3
Метод	getDate()	Возвращает день месяца (от 1 до 31).
	getDay()	Возвращает день недели (от 0 до 6).
	getFullYear()	Возвращает год в четырех символах по местному времени.
	getHours()	Возвращает час суток (от 0 до 23).
	getMilliseconds()	Возвращает миллисекунды.
	getMinutes()	Возвращает минуты в пределах часа (от 0 до 59)
	getMonth()	Возвращает месяцы года (от 0 до 11)
	getSeconds	Возвращает секунды (от 0 до 59)
	getTime	Возвращает количество миллисекунд, начиная с 1/1/1970 00:00:00
1	2	3
	getTimeZoneOffset()	Возвращает смещение часового пояса в минутах по отношению к GMT/UTC
	getUTCDate()	Возвращает день месяца

	getUTCDay()	Возвращает день недели, преобразованный к универсальному времени.
	getUTCFullYear()	Возвращает четырехзначное представление года, преобразованное к универсальному времени.
	getUTCHours()	Возвращает часы, преобразованные к универсальному времени.
	getUTCMilliseconds()	Возвращает миллисекунды, преобразованные к универсальному времени.
	getUTCMinutes()	Возвращает минуты, преобразованные к универсальному времени.
	getUTCMonth()	Возвращает месяц, преобразованный к универсальному времени.
	getUTCSeconds	Возвращает секунды, преобразованные к универсальному времени.
	getYear()	Возвращает номер года, начиная с 1900 г.
	parse()	Преобразует строковые данные в миллисекунды.
	setDate()	Устанавливает день месяца (от 1 до 31).
	setFullYear()	Устанавливает год как четырехзначное число.
	setHours()	Устанавливает часы дня (от 0 до 23).
	setMilliseconds()	Устанавливает миллисекунды.
	setMinutes()	Устанавливает минуты в часе (от 0 до 59).
	setMonth()	Устанавливает месяц в году (от 0 до 11).
	setSeconds	Устанавливает секунды в минуте (от 0 до 59).
	setTime()	Устанавливает количество миллисекунд, начиная с 1/1/1970 00:00:00
	setUTCDate()	Устанавливает день месяца в соответствии с универсальным временем.
	setUTCFullYear()	Устанавливает год как четырехзначное число в соответствии с универсальным временем.
	setUTCHours()	Устанавливает час в соответствии с универсальным временем.
	setUTCMilliseconds()	Устанавливает миллисекунды в соответствии с универсальным временем.
	setUTCMinutes()	Устанавливает минуты в соответствии с универсальным временем.
	setUTCMonth()	Устанавливает месяц в соответствии с универсальным временем.
	setUTCSeconds()	Устанавливает секунды в соответствии с универсальным временем.
	setYear()	Устанавливает количество лет, начиная с 1900 г.
	toGMTString()	Преобразует дату в строку в соответствии с мировым форматом.
	toLocalString()	Возвращает строку даты в формате локальной системы.
	toSource()	Возвращает источник-объект date.
	toString()	Возвращает дату и время как строку в соответствии с местным временем.
	ToUTCString()	Возвращает дату и время как строку в соответствии с мировым временем (UTD).
	UTC()	Преобразует значения с разделителями-запятыми в миллисекунды даты по UTC.
	valueOf()	Возвращает эквивалент объекта Date в миллисекундах.
Свойство	prototype	Свойство, позволяющее добавлять методы и свойства к объекту Date.

Ниже представлен сценарий вывода в HTML-документ текущей даты в **международном формате**.

Листинг 2.1 Вывод в HTML-документ текущей даты в международном формате

```
<!DOCTYPE html>
<html>
<head>
<title>Data </title>
</head>
<body>
<script>
var theDate=new Date()
document.write(theDate)
</script>
</body>
</html>
```

Далее представлен сценарий, выводящий текущую дату и время в HTML-документ.

Листинг 2.2 Вывод текущей даты и время в HTML-документ

```
<!DOCTYPE html>
<html>
<head>
<title>Data </title>
</head>
<body>
<script>
var theDate=new Date()
document.write("<table cellpadding=5 width=100%
border=0>"+<tr><td width=95% bgcolor=gray align=left>"+<font
color=red>Date:"+theDate+"</font></td></tr></table>")

</script>
</body>
</html>
```

Пример сценария изменения фона от времени суток.

Листинг 2.3 Изменения фона от времени суток

```
<!DOCTYPE html>
<html>
<head>
<title>Времясуток</title>
</head>

<body>
<script>
theTime=new Date()
theHour=theTime.getHours()
if(18>theHour)
document.writeln("<body bgcolor=white text='Black'>")
else
```

```
document.writeln("<body bgcolor=black text='White'>")
</script>
</body>
</html>
```

Разработайте сценарий, по которому в зависимости от поры года выводилась бы соответствующая картинка.

Вывод реального времени демонстрирует следующий сценарий.

Листинг 2.4 Вывод реального времени

```
<!DOCTYPE html>
<html>
<head>
<title>Вывод реального времени</title>
</head>

<body>
<script>
now=new Date()
document.write("Time:"+now.getHours()+":"+now.getMinutes()+"<br
>")
document.write("Date:"+ (now.getMonth()+1) + "/" + now.getDate() + "/" +
now.getYear())
</script>
</body>
</html>
```

В сценарии, записанном ниже, реализованы идущие часы.

Листинг 2.5 Идущие часы

```
<!DOCTYPE html>
<html>
<head>
<title>идущие часы</title>

<script>

var ch=1,m=2,s=3
function clock(){
now=new Date()
ch=now.getHours()
m=now.getMinutes()
s=now.getSeconds()
window.document.forms[0].elements[0].value=ch+":"+m+":"+s
setTimeout("clock()",1000)
}

</script>
</head>
<body onLoad="clock()">
<form name="form1">
```

```

<input type="text">
</form>
</body>
</html>

```

Измените рассмотренный выше сценарий таким образом, чтобы идущие часы выводились в формате 00:00:00.

Разработайте сценарий «Планер» на июнь месяц с учетом планируемых дел.

1.3 Объект Math

Объект Math является встроенным объектом языка JavaScript и содержит свойства и методы, используемые для выполнения математических операций. Объект Math включает также некоторые широко применяемые математические константы.

Синтаксис:

```

Math.propertyName
Math.methodName(parameters)

```

Методы и свойства объекта Date представлены в таблице 3.

Таблица 3 – Методы и свойства объекта Math.

Тип	Элемент	Описание
1	2	3
Метод	abs()	Возвращает абсолютное значение.
	acos()	Возвращает значение арккосинуса для аргумента, выраженное в радианах.
	asin()	Возвращает значение арксинуса для аргумента, выраженное в радианах.
	atan()	Возвращает значение арктангенса для аргумента, выраженное в радианах.
	atan2()	Возвращает значение арктангенса для частного от деления аргументов, выраженное в радианах.
	ceil()	Возвращает целое число, большее или равное переданному значению.
	cos()	Косинус для переданного значения.
	exp()	Возвращает константу Эйлера для степени, переданной в качестве значения.
	floor()	Целое число, меньшее или равное переданному значению.
1	2	3
	log()	Натуральный логарифм по основанию e.
	max()	Максимальное число из двух переданных значений.
	min()	Минимальное число из двух переданных значений.
	pow()	Результат возведения первого переданного в качестве аргумента числа в степень второго.
	random()	Возвращает случайное число между 0 и 1.
	round()	Возвращает значение, округленное до ближайшего целого.
	sin()	Возвращает синус переданного числа, выраженный в радианах.
	sqrt()	Квадратный корень переданного числа.
	tan()	Возвращает тангенс переданного числа, выраженный в радианах.
	toSource()	Копия объекта.
	toString()	Строковое представление объекта.

Свойства	E	Константа Эйлера (2.718281828459045).
	LN2	Натуральный логарифм 2 (0.6931471805599453).
	LN10	Десятичный логарифм 10 (2.302585092994046).
	LOG2E	Логарифм e по основанию 2 (1.4426950408889633).
	LOG10E	Логарифм e по основанию 10 (0.4342944819032518).
	PI	Число Пи (3.141592653589793).
	SORT1 2	Квадратный корень 0.5 (0.7071067811865476).
	SORT2	Квадратный корень 2 (1.4142135623730951).

Пример сценария вычисления площади круга без округления представлен ниже.

Листинг 3.1 Вычисление площади круга без округления

```
<!DOCTYPE html>
<html>
<head>
<title>Вычисление площади круга без округления</title>
<script>
function square(n) {
var result=0
result=n*n
}
function square1(n) {
var result=0
result=n*n
return result
}
function square2(n) {
return n*n
}
</script>
</head>
<body>
<script>
document.write (square(3))
document.write ("<br>")
document.write (square1(5))
document.write ("<br>")
document.write (square2(7))
document.write ("<br>")
document.write ("<table border='1'
width='600'><tr><th>число</th><th>площадь круга</th></tr>")
for(var i=0;i<100;i++) {
document.write
("<tr><th>",i,"</th><th>",square1(i)*Math.PI,"</th></tr>")
}
document.write ("</table>")
</script>
</body></html>
```

Далее предлагается ознакомиться со сценарием вычисления площади круга с округлением результата до сотых.

Листинг 3.2 Вычисление площади круга с округлением результата до сотых


```

<!DOCTYPE html>
<html>
<head>
<title>Округлением результата до сотых</title>
<script>
function square2(n) {
return n*n
}
function round1(n){
returnMath.round(n*100)/100
}
</script>
</head>
<body>
<script>
document.write("<table border='1'
width='600'><tr><th>число</th><th>площадь  круга</th></tr>")
for(var i=0;i<100;i++) {
document.write
("<tr><th>",i,"</th><th>",round1(square2(i)*Math.PI),"</th></tr>");/
/округление до ближайшего целого
}
document.write("</table>")
</script>
</body>
</html>

```

Измените сценарий вычисления площади круга с округлением результата до k знаков после запятой.

Разработайте сценарий вывода в HTML-документ случайных чисел в диапазоне от m до n.

1.4 Строковые объекты

Строка (string) в языке JavaScript представляется в виде последовательности символов, заключенных в двойные или одинарные кавычки. Для управления строковыми объектами используется следующий синтаксис

```

stringName.propertyName
stringName.methodName(parameters)

```

Имеется в виду, чтоstringName – это имя объекта String. Строки можно создавать тремя способами:

- 1) создать строковую переменную при помощи оператора var и присвоить ей строковое значение;
- 2) присвоить значение строковой переменной только посредством оператора присваивания (=);
- 3) использовать конструктор String().

Свойства. Значением свойства length является длина строки. Например, выражение “Script”.length вернет значение 6, поскольку строка “Script” содержит 6 символов.

Методы объекта String представлены в таблице 4.

Таблица 4 – Методы объекта String

Метод	Описание метода
anchor()	Создает экземпляр дескриптора <a> с атрибутом name, принимающим значение строки, передаваемой методу.
big()	Аналогично тегам HTML<big>...</big> позволяет отобразить более крупным шрифтом.
blink() (не работает)	Заставляет строку мигать. (Этим методом почти никто не пользуется)
bold()	Название говорит само за себя – делает символы жирными.
charAt(arg)	Возвращает символ, находящийся в заданной позиции строки. Пример: vpos="Script".charAt(3); переменной vpos будет присвоено значение "r".
CharCodeAt()	Возвращает код символа ISO-Latin-1, расположенного по индексу, переданному в метод.
concat(t)	Связывает две переменные строки в одну новую.
fixed()	Аналогично <tt>...</tt> вывод строки фиксированного размера.
fontcolor(arg)	Аналогично.... Аргумент метода может быть как триплетом RGB, так и зарезервированным словом.
fontsize(arg)	Позволяет изменять размер шрифта. Аргумент в условных единицах. Аналогично.... Также можно использовать вид +size или -size для увеличения или уменьшения шрифта на size единиц, например, "строк a".fontsize(+1).
fromCharCode();	Возвращает строковое значение числа 'ISO-Latin-1', переданное методу.
indexOf(arg1[,arg2])	Возвращает позицию в строке, где впервые встречается символ – arg1 , необязательный числовой аргумент arg2 указывает начальную позицию для поиска.
italics	Аналогично тегам HTML<i>...</i> позволяет отобразить италик
lastIndexOf(arg1[,arg2])	Возвращает либо номер позиции в строке, где в последний раз встретился символ - arg1, либо -1, если символ не найден. Arg2 задает начальную позицию для поиска.
link()	Аналогично тегам HTML<a href>... позволяет преобразовать строку в гиперсвязь.
match()	Возвращает массив, основанный на регулярном выражении, переданном в метод.
replace()	Выполняет операцию поиска и замены, заданную регулярным выражением, и заменяет на строку, переданную в метод.
search()	Возвращает позицию совпадения с заданной строкой в экземпляре объекта String. Если строка не найдена, возвращается значение -1.
slice()	Возвращает часть строки между заданными начальной и конечной позициями. При передаче отрицательного числа отсчет будет выполняться с конца строки!
small()	Аналогично тегам ... позволяет отображать строку мелким шрифтом.
split()	Выполняет разделение строки на сегменты. Размеры сегментов определяются границами строки и экземпляров.
strike()	Аналогично тегам <strike>...</strike> позволяет отображать строку зачеркнутой.
sub()	Аналогично тегам ^{...} позволяет отображать строку нижним индексом.
substr()	Возвращает часть строки, которая начинается с заданной позиции и содержит заданное количество символов. При передаче отрицательного числа отсчет будет выполняться с конца строки.

substring(arg1,arg2)	Позволяет извлечь подстроку длиной arg2, начиная с позиции arg1.
sup()	Аналогично тегам ^{...} позволяет отображать строку верхним индексом.
toLowerCase()	Преобразует символы строкового объекта в строчные.
toUpperCase()	Преобразует символы строкового объекта в прописные.

Далее предлагается понять и проверить сценарии JavaScript с использованием объекта String в различных браузерах. Предложенное задание имеет статус дополнительного.

Листинг 4.1 Движущийся текст в блочном элементе

Допишите нужный текст сценария.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Document</title>
  <script>
// Задаем выводимый текст
var scrtxt="Мы на многае не адважваемся не таму, што яно складанае;" +
" яно складанае менавіта таму, што мы на яго не адважваемся." +
" (Сенека (4 да н.э. – 65 н.э.), рымскі палітычны дзеяч, філосаф і
пісьменнік)" +
"Недастаткова, каб выхаванне толькі не псавала нас," + " трэба, каб яно
мяняла нас у лепшы бок. " +
"(Мішэль дэ Мантэнь (1533-1592), французскі філосаф)";

var _length=scrtxt.length;
var _width=60;
var pos=-(_width+2);
function scroll() {
  // выводим текст с правой позиции и устанавливаем временную задержку
  // сдвигаем позицию на один шаг
  pos++;
  // вычисляем, какая часть текста будет видима
  var scroller="";
  if (pos==_length) {
    pos=-(_width+2);
  }
  // если текст не достиг пока левого края,
  // то необходимо добавить несколько пробелов - иначе придется
  //отсекать переднюю часть текста
  if (pos<0) {
    for (var i=1;i<=Math.abs(pos);i++) {
      scroller=" "+scroller;
    }
    scroller=scrtxt.substring(0,_width-i+1)+scroller;
  }
  else {
```

```

scroller=sqrtxt.substring(pos,_width+pos)+scroller;
}
// выводим текст в блочный элемент
Дописать нужную строку
// вновь обращаемся к функции спустя 100 миллисекунд
setTimeout("scroll()",100);
}
</script>
<body onload="scroll()">

```

Любой блочный элемент

```

</body>
</html>

```

Листинг 4.2 Поиск заданной буквы в заданном тексте.

Исправьте синтаксическую и логическую ошибки.

```

<!DOCTYPE html>
<head>
<title>Замена</title>
<script>
varmyString="Поиск заданной буквы в заданном тексте"
function findLetter (txt,letter){
varpos=0
varnum=0
vari=0
while (pos!=-1){pos=txt.indexOf(letter,i)
if(pos!=-1) num=num+1
i=pos+1
return num
}
}
</script>
</head>
<body>
<script>
document.write("<br>", findLetter (myString,"y"))
</script>
</body>
</html>

```

Разработайте сценарий JavaScript замены в тексте буквы «а» на букву «б».

Получите у преподавателя индивидуальное задание. Разберите условие задач.